Setor de Educação Profissional e Tecnológica
Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

14 - Caminhos mínimos

Prof. Alex Kutzke

Novembro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Definir caminho mínimo e distância em um grafo com pesos, e mostrar com um exemplo por que a busca em largura não os encontra;
2. Executar à mão o algoritmo de Dijkstra, registrando as distâncias, a fila de prioridade e os candidatos descartados;
3. Explicar a relaxação de uma aresta e por que o algoritmo exige pesos não negativos;
4. Justificar o custo do algoritmo;
5. Diferenciar a árvore de caminhos mínimos da árvore geradora mínima.

Este conteúdo cai na Prova 2.

Roteiro

O problema

Dijkstra

Fechamento

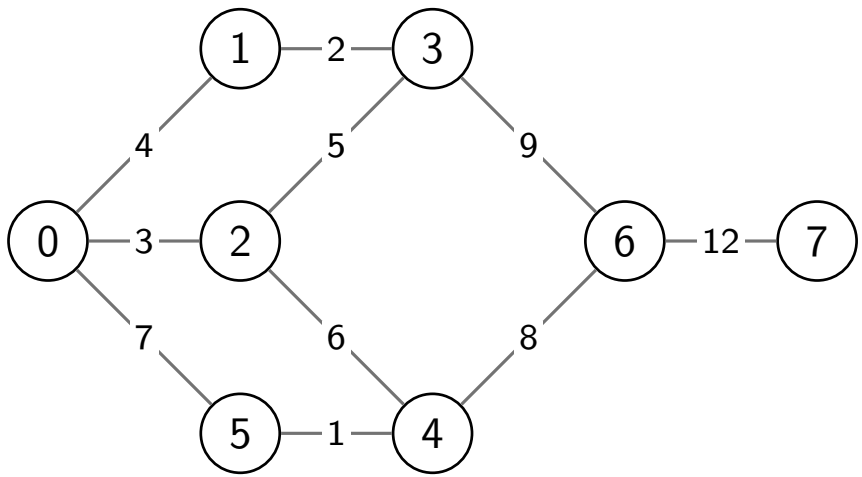
Demonstração: rotas até as escolas

```
$ go run rotas.go
Árvore geradora mínima                  Árvore de caminhos mínimos, de S
|            .K....        ...G   |   |             K            ...G   |
|   A..   ..       ...F...         |   |   A..         .      F...        |
|      ..B.                        |   |      ..B.     .      .           |
|          ..                      |   |           ..  .  ..              |
|            ..                    |   |             ... .                |
|              .S...               |   |               .S...              |
|              .    ....H          |   |                . .  ....H        |
|     C.       .           ..      |   |     C.         .  .        ..    |
|   ..  ...    .              ..   |   |        ...     .     .         ..  |
| D.        .E...              .I  |   | D.........E      .            .I  |
|               ...J               |   |                    J             |
81.6 km de estrada                      95.1 km de estrada
```

- as mesmas 21 estradas e as mesmas 12 localidades, com duas perguntas diferentes;
- da sede até a escola F: 23,1 km pela árvore geradora mínima e 9,4 km pelo caminho mínimo.

Caminho mínimo

- **peso de um caminho**: soma dos pesos das arestas;
- **caminho mínimo** de s a v: caminho de menor peso entre eles;
- **distância** de s a v: o peso do caminho mínimo.



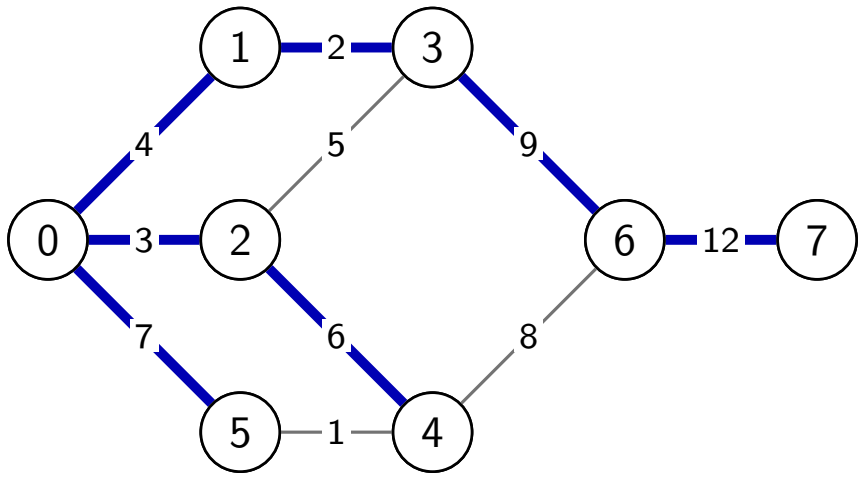
Em G, de 0 a 6:

- 0-1-3-6: $4 + 2 + 9 = 15$;
- 0-5-4-6: $7 + 1 + 8 = 16$;
- 0-2-4-6: $3 + 6 + 8 = 17$.

A busca em largura não olha os pesos

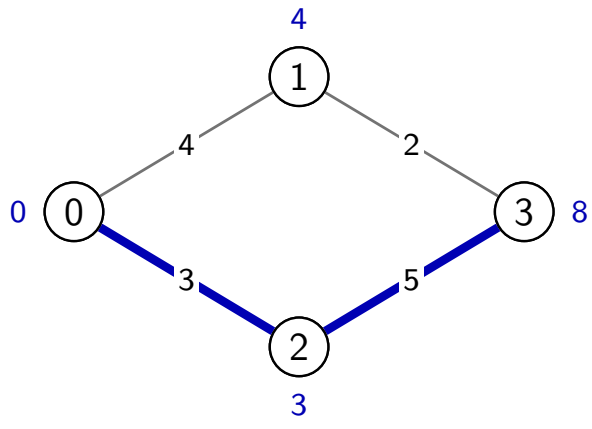
- a busca em largura minimiza o número de arestas;
- na árvore ao lado, ela chega ao 4 por 0-2-4, com peso 9;
- 0-5-4 também tem duas arestas, e o peso é 8.

Com pesos, o próximo vértice a visitar depende da distância percorrida até ele.

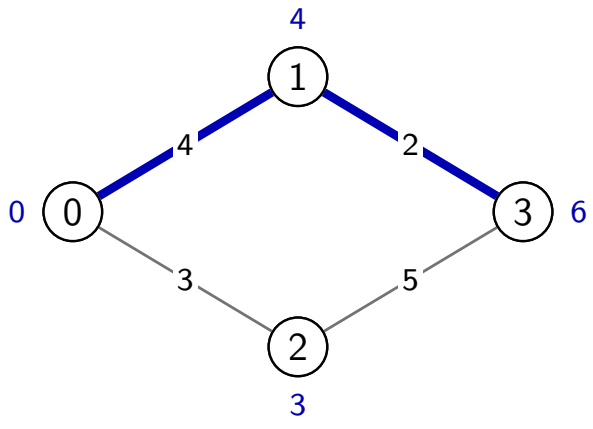


Relaxação de uma aresta

O vetor `dist` guarda o peso do melhor caminho conhecido até cada vértice, e o vetor `anterior`, o vértice que vem antes dele nesse caminho.



dist[3] = 8, com anterior[3] = 2



4 + 2 < 8: dist[3] = 6, anterior[3] = 1

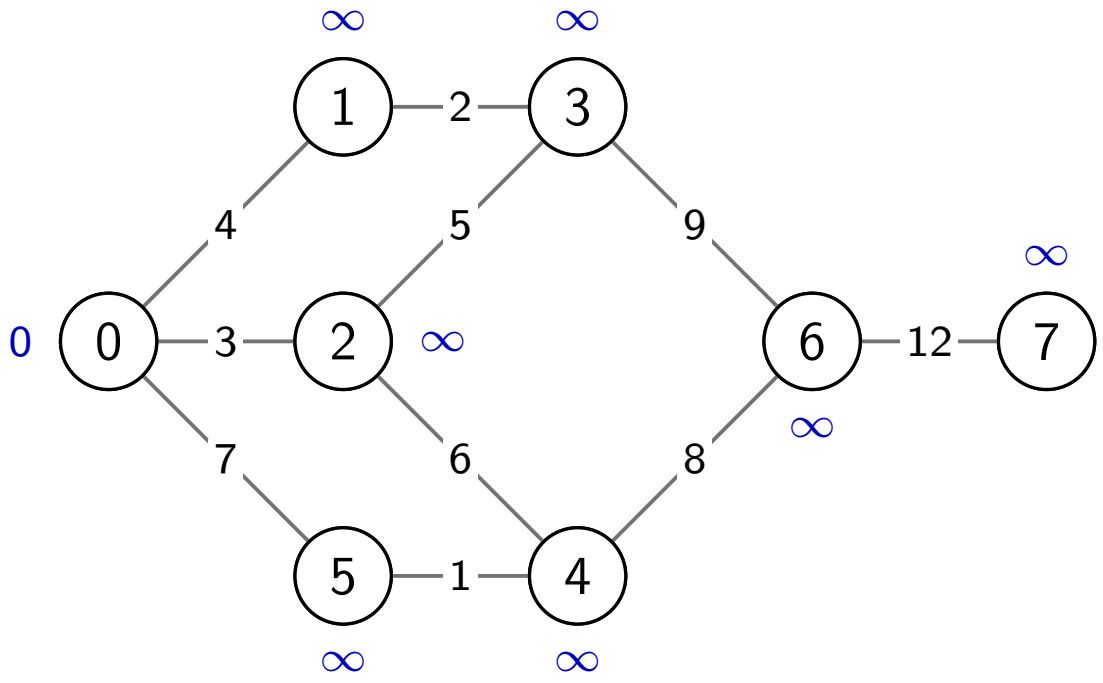
```
if dist[v]+a.Peso < dist[w] { // passar por v melhora o caminho até w?
    dist[w] = dist[v] + a.Peso
    anterior[w] = v
}
```

Algoritmo de Dijkstra

1. Comece com `dist[s]` $= 0$, as outras distâncias infinitas e s na fila de prioridade;
2. Retire da fila o candidato de menor distância;
3. Se o vértice já está na árvore, descarte o candidato;
4. Senão, coloque o vértice na árvore e relaxe as arestas dele. Cada melhora põe um novo candidato na fila;
5. Repita até a fila esvaziar.

Um vértice pode ter vários candidatos na fila, um para cada melhora da distância. Só o menor deles coloca o vértice na árvore.

Dijkstra passo a passo



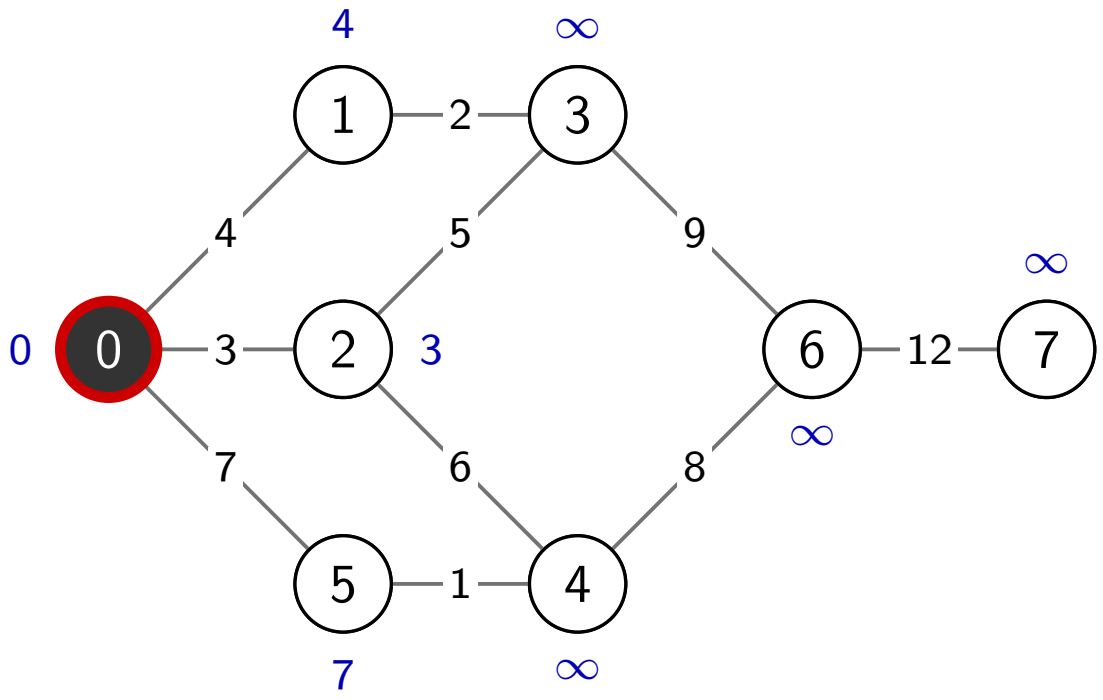
fila de prioridade

0 (0)

início: a distância de 0 é 0, e as outras são infinitas

na árvore entra agora candidato descartado

Dijkstra passo a passo



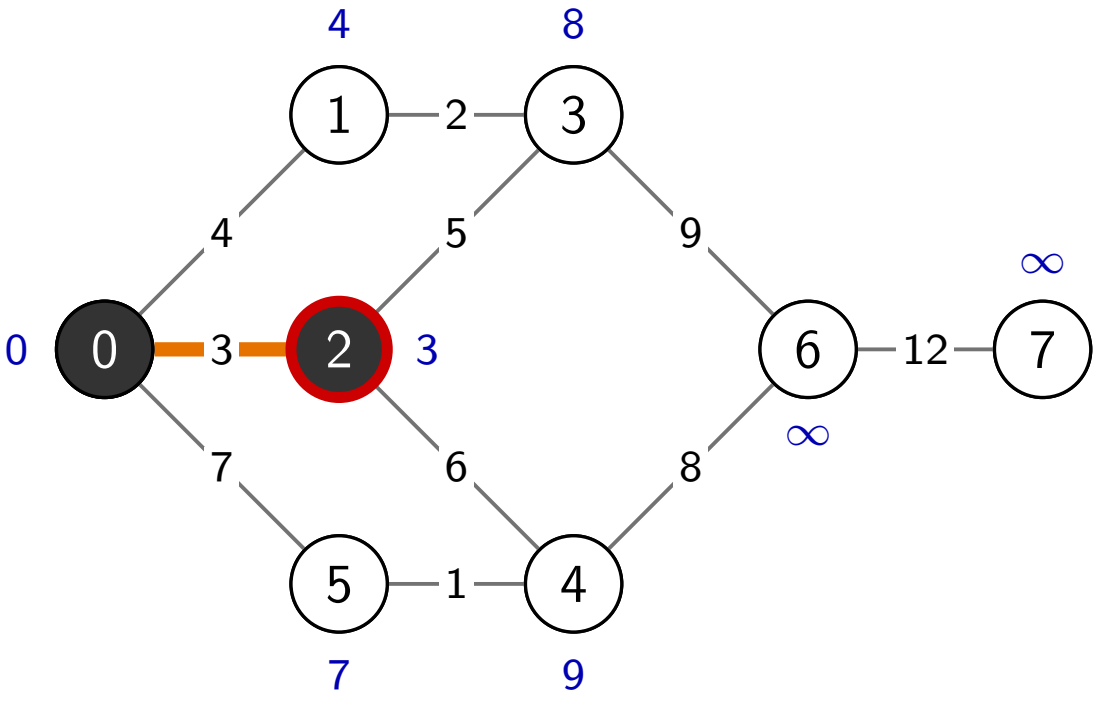
fila de prioridade

2 (3)
1 (4)
5 (7)

retira 0(0): 0 entra; relaxa 0-1, 0-2 e 0-5

na árvore — entra agora — candidato descartado

Dijkstra passo a passo



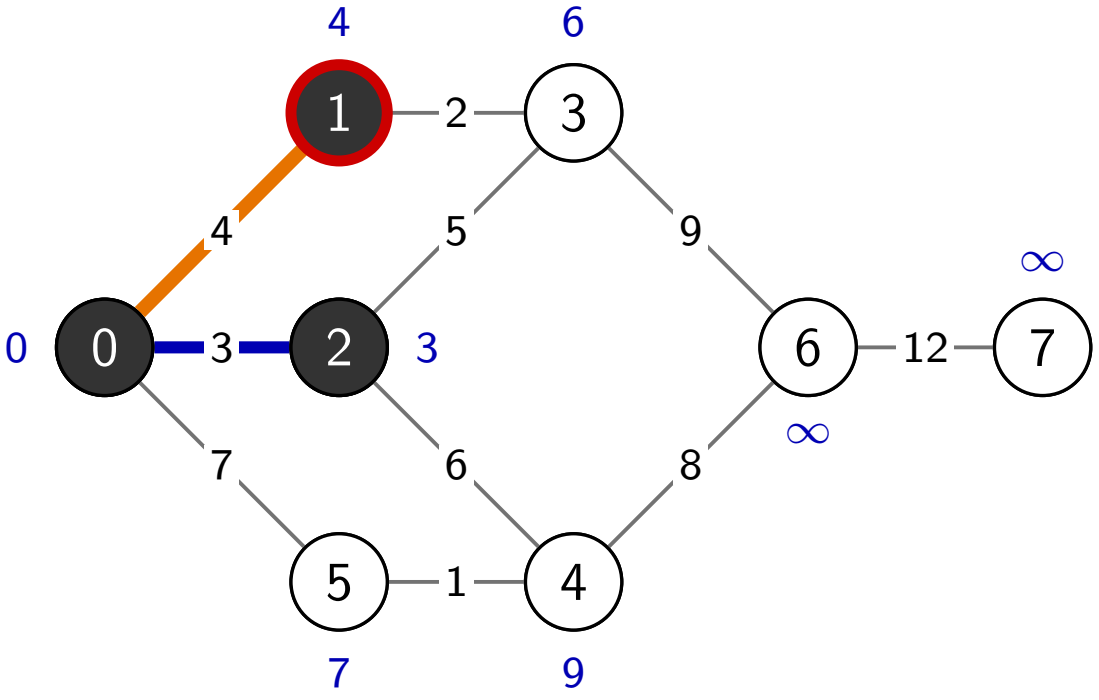
fila de prioridade

1 (4)
5 (7)
3 (8)
4 (9)

retira 2(3): 2 entra; relaxa 2-3 e 2-4

na árvore — entra agora — candidato descartado

Dijkstra passo a passo



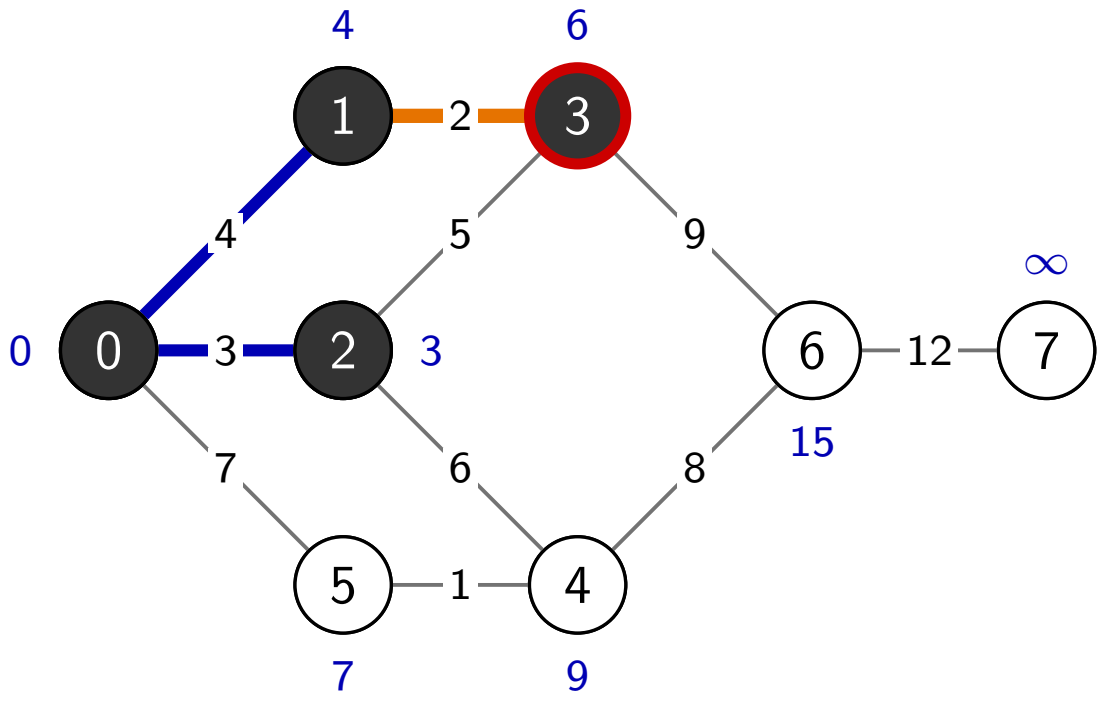
fila de prioridade

3 (6)
5 (7)
3 (8)
4 (9)

retira 1(4): 1 entra; relaxa 1-3, e 6 é menor que 8

na árvore — entra agora — candidato descartado

Dijkstra passo a passo



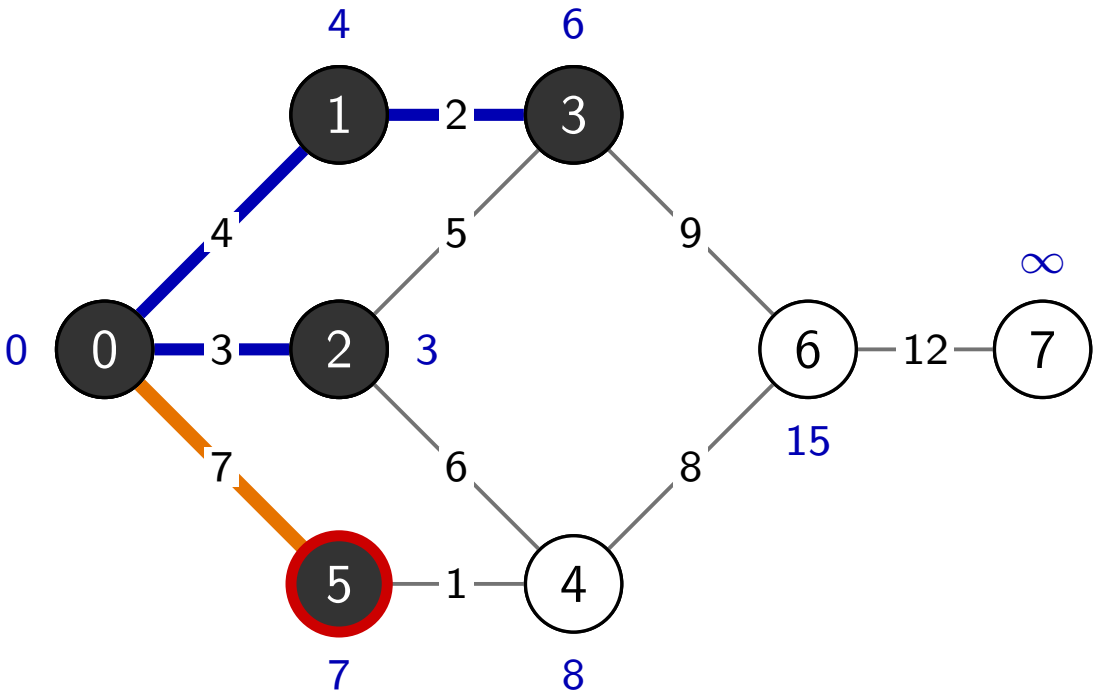
fila de prioridade

5 (7)
3 (8)
4 (9)
6 (15)

retira 3(6): 3 entra; relaxa 3-6

na árvore — entra agora — candidato descartado

Dijkstra passo a passo



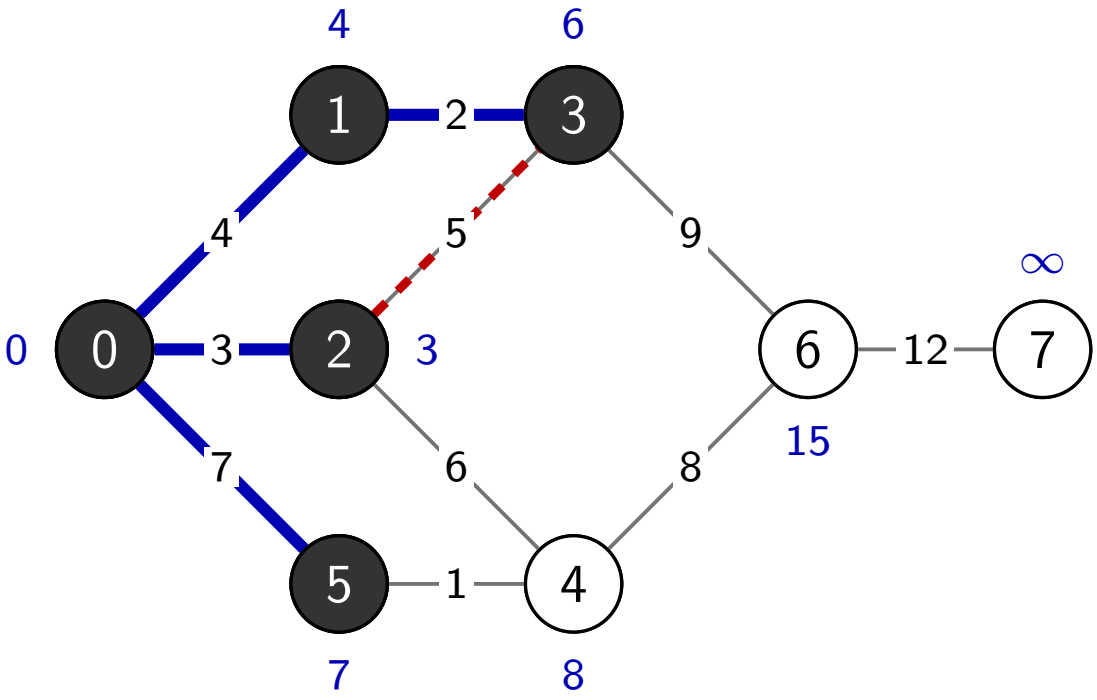
fila de prioridade

3 (8)
4 (8)
4 (9)
6 (15)

retira 5(7): 5 entra; relaxa 5-4, e 8 é menor que 9

na árvore — entra agora — candidato descartado

Dijkstra passo a passo



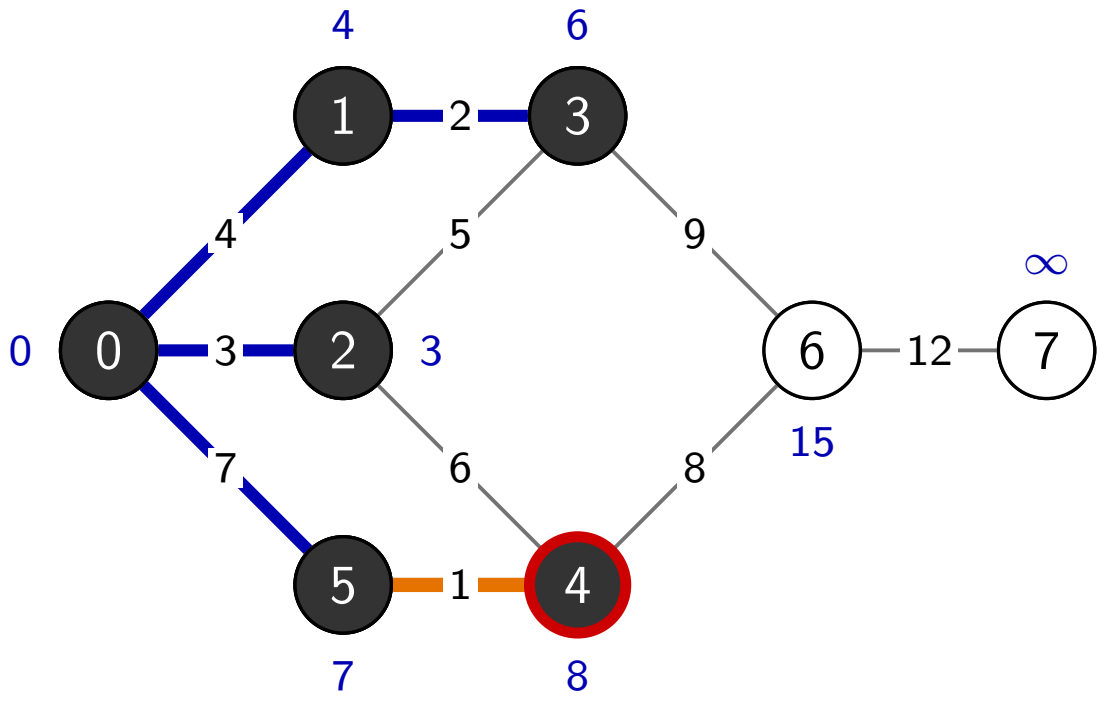
fila de prioridade

4 (8)
4 (9)
6 (15)

retira 3(8): 3 já está na árvore, com distância 6; descarta



Dijkstra passo a passo



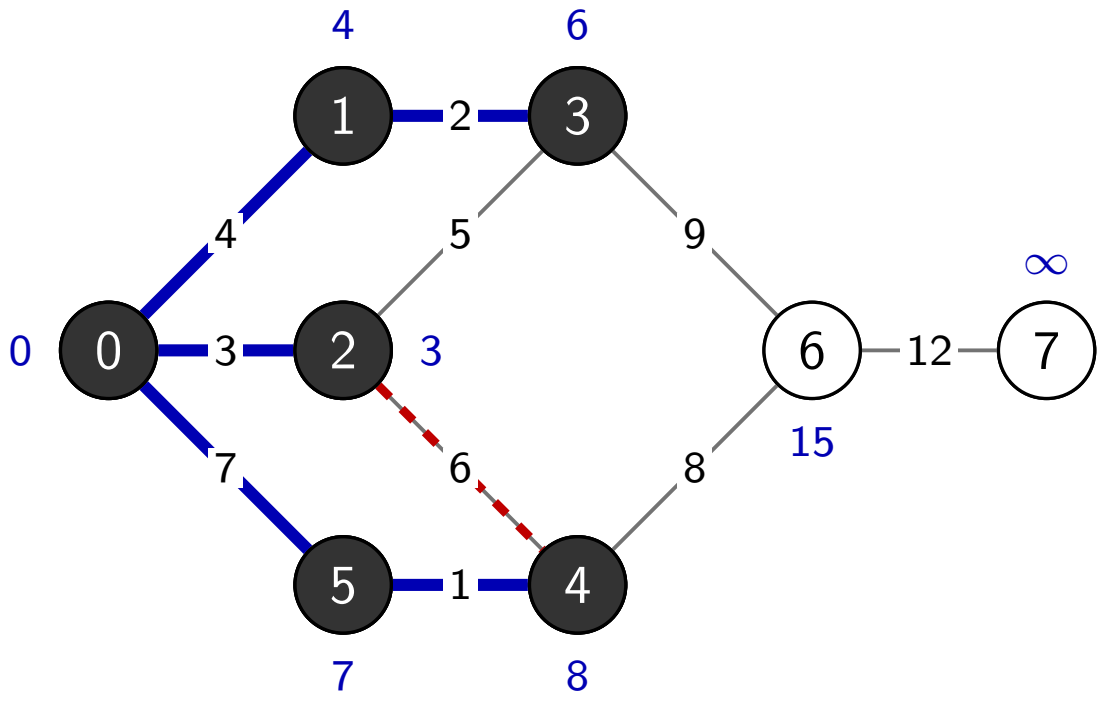
fila de prioridade

4 (9)
6 (15)

retira 4(8): 4 entra; 4-6 daria 16, que não melhora 15

na árvore — entra agora — candidato descartado

Dijkstra passo a passo



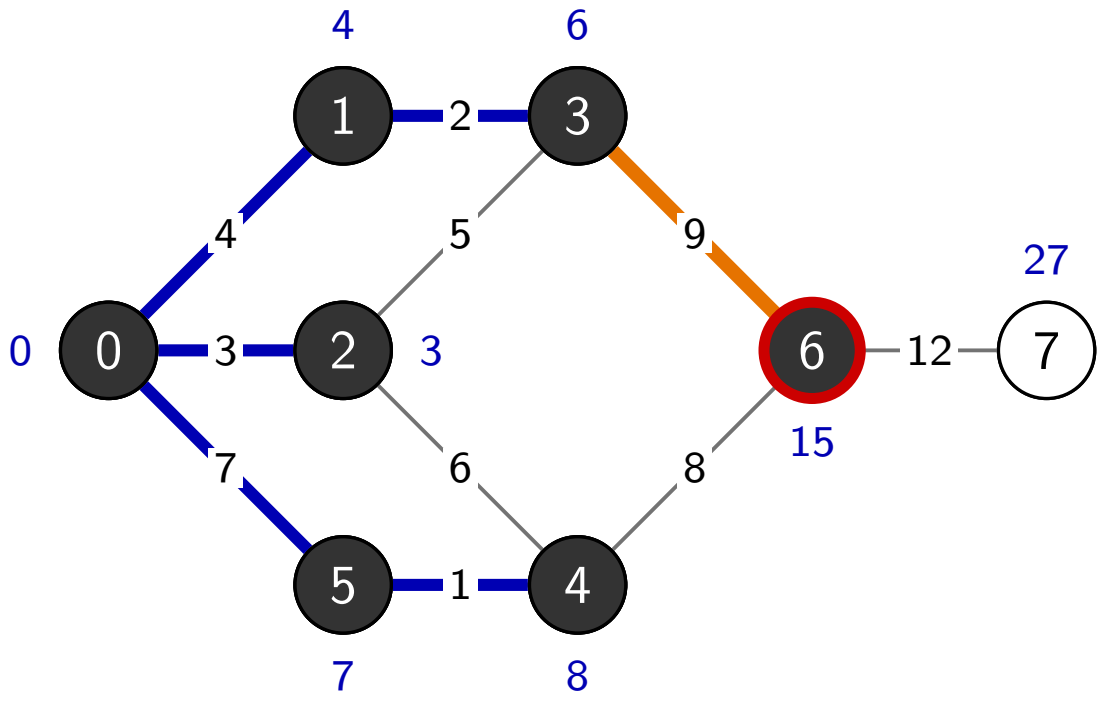
fila de prioridade

6 (15)

retira 4(9): 4 já está na árvore, com distância 8; descarta

na árvore — entra agora — candidato descartado

Dijkstra passo a passo



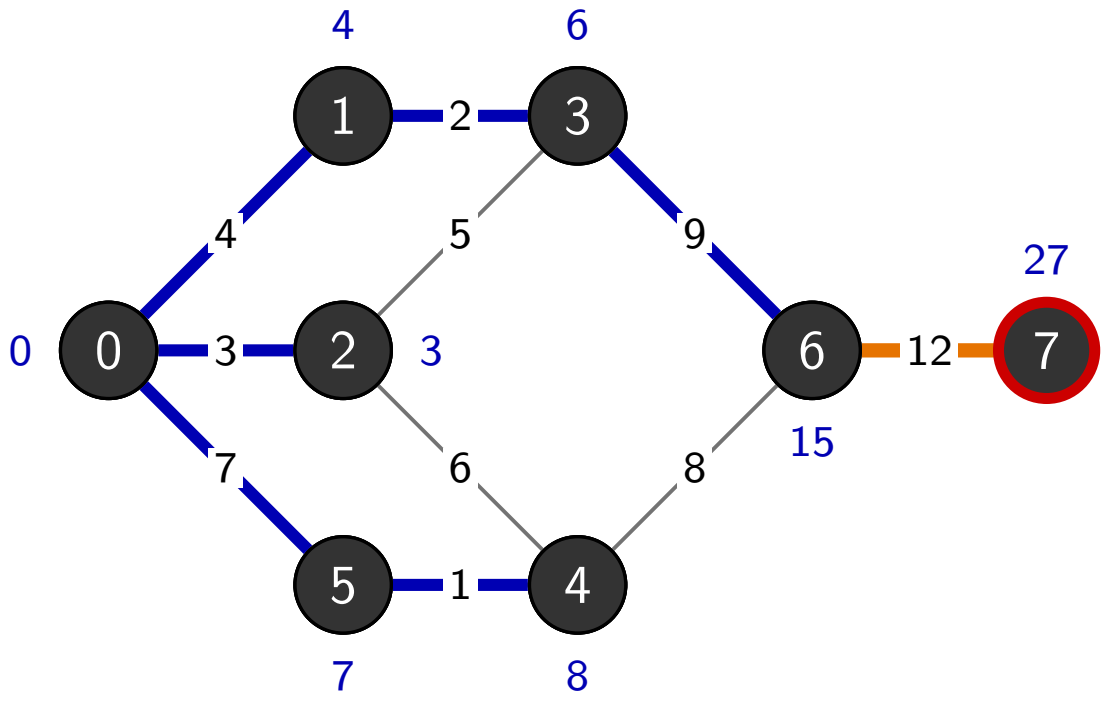
fila de prioridade

7 (27)

retira 6(15): 6 entra; relaxa 6-7

na árvore entra agora candidato descartado

Dijkstra passo a passo



fila de prioridade

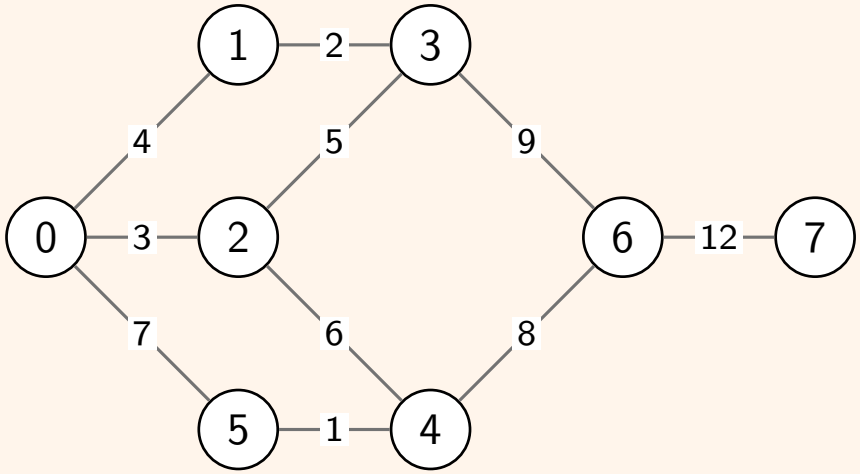
retira 7(27): 7 entra; a fila ficou vazia, fim

na árvore entra agora candidato descartado

Prática: Dijkstra a partir do vértice 3

Tempo: 5 minutos. Em dupla, no papel:

- execute o algoritmo em G a partir do vértice 3;
- anote a ordem em que os vértices entram na árvore;
- anote a distância de cada vértice;
- identifique o candidato descartado.



Resposta: Dijkstra a partir do vértice 3

Ordem de entrada: 3, 1, 2, 0, 6, 4, 5, 7.

vértice	0	1	2	3	4	5	6	7
`dist`	6	2	5	0	11	12	9	21
`anterior`	1	3	3	-	2	4	3	6

- quando o 0 entra, com distância 6, a aresta 0-5 põe na fila o candidato 5(13);
- quando o 4 entra, com distância 11, a aresta 4-5 põe o candidato 5(12), que sai primeiro;
- o candidato 5(13) sai depois, com o 5 já na árvore, e é descartado.

Dijkstra em Go: candidatos e início

```go
// Candidato é um vértice que pode entrar na árvore, com a distância da
// origem até ele pelo caminho encontrado até aqui.
type Candidato struct {
    V    int
    Dist float64
}

func Dijkstra(g *GrafoP, s int) (dist []float64, anterior []int) {
    dist = make([]float64, g.V())
    anterior = make([]int, g.V())
    for v := range dist {
        dist[v] = math.Inf(1) // ainda sem caminho
        anterior[v] = -1
    }
    naArvore := make([]bool, g.V())
    fila := &FilaCandidatos{}
```

`FilaCandidatos` tem a forma de `FilaArestas` do Prim, com `Dist` como prioridade.

Dijkstra em Go: laço principal

```
    dist[s] = 0
    heap.Push(fila, Candidato{s, 0})
    for fila.Len() > 0 {
        c := heap.Pop(fila).(Candidato)
        v := c.V
        if naArvore[v] {
            continue // v já entrou por um caminho menor
        }
        naArvore[v] = true
        for _, a := range g.adj[v] {
            w := a.Outro(v)
            if !naArvore[w] && dist[v]+a.Peso < dist[w] {
                dist[w] = dist[v] + a.Peso
                anterior[w] = v
                heap.Push(fila, Candidato{w, dist[w]})
            }
        }
    }
    return dist, anterior
}
```

Por que o menor candidato já é a distância

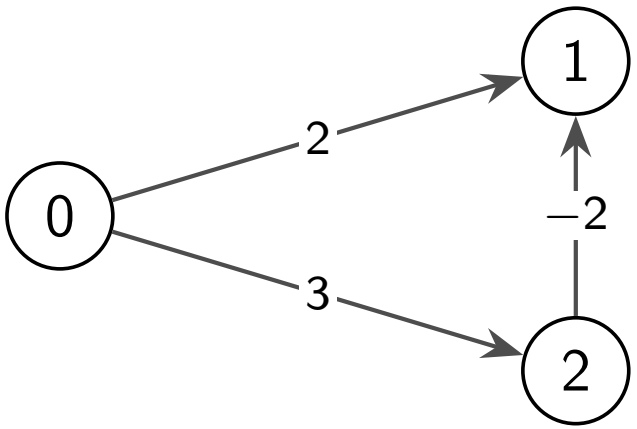
- o candidato v tem a menor distância da fila, d;
- qualquer outro caminho de s a v precisa sair da árvore por alguma aresta x-y, com x na árvore e y fora;
- até y, esse caminho pesa pelo menos `dist[y]`, e o candidato de y na fila tem distância maior ou igual a d;
- de y até v, os pesos não são negativos, e o caminho inteiro pesa pelo menos d.

Em G, a partir de 0, o vértice 3 sai da fila com distância 6. Os outros candidatos têm distância 7 ou mais, e nenhum caminho que passe por eles chega ao 3 com peso menor.

Pesos negativos

- o vértice 1 entra na árvore com distância 2;
- depois, o vértice 2 entra com distância 3, e a aresta 2-1 daria $3 + (-2) = 1$;
- o 1 já está na árvore, e a distância fica errada: 2 em vez de 1.

Em grafo não dirigido, uma aresta negativa é um ciclo: ir e voltar por ela reduz o peso sem limite. Pesos negativos pedem outro algoritmo, como o de Bellman-Ford, que não é tratado nesta disciplina.

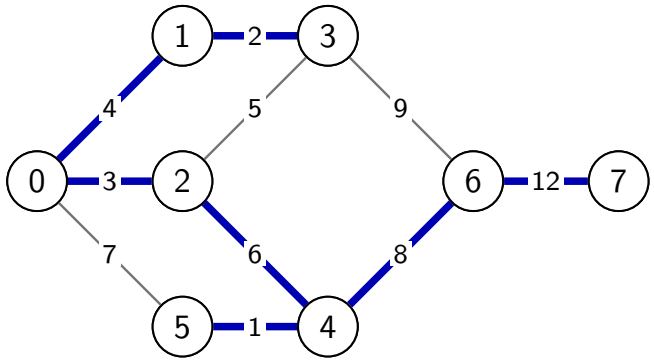


Custo do Dijkstra

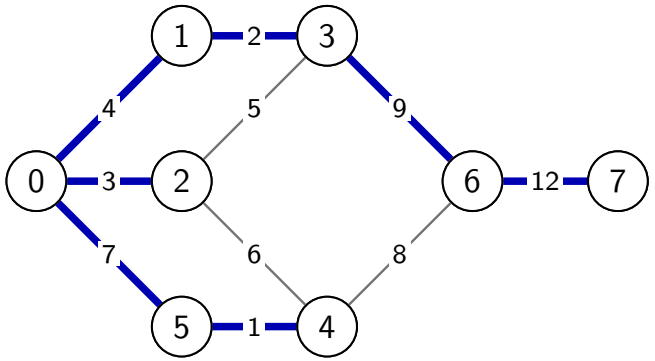
- uma aresta só é relaxada com sucesso quando o primeiro dos seus extremos entra na árvore;
- a fila recebe no máximo $E + 1$ candidatos, e cada operação custa $O(\log E)$;
- total: $O(E \log E)$, igual a $O(E \log V)$, porque $\log E \leq 2 \log V$;
- em grafo denso, um vetor no lugar da fila dá $O(V^2)$, sem heap.

Com todas as arestas de mesmo peso, a busca em largura encontra o caminho mínimo em $O(V + E)$, sem fila de prioridade.

Dijkstra e Prim



árvore geradora mínima: peso 36



caminhos mínimos de 0: peso 38

	Prim	Dijkstra
Prioridade na fila	peso da aresta	distância da origem
Resultado	árvore geradora mínima	árvore de caminhos mínimos
Depende da origem	não, com pesos distintos	sim
Custo	$O(E \log E)$	$O(E \log E)$

Atividade Avaliativa 4

- o restante do encontro é da Atividade Avaliativa 4, com o enunciado e o esqueleto na UFPR Virtual;
- depois: exercícios do roteiro da aula, sem entrega e sem nota. O conteúdo cai na Prova 2;
- rodar `dijkstra.go` e `rotas.go`, trocando as arestas e as estradas;
- leitura: Sedgewick, seção 4.4; Cormen, capítulo 24.

Próxima aula, 18/11: livre para trabalho prático e dúvidas, presença facultativa, Prova 2 em 25/11.